

Competitive Programming

Section 37: Segment Trees

Competitive Programming Course

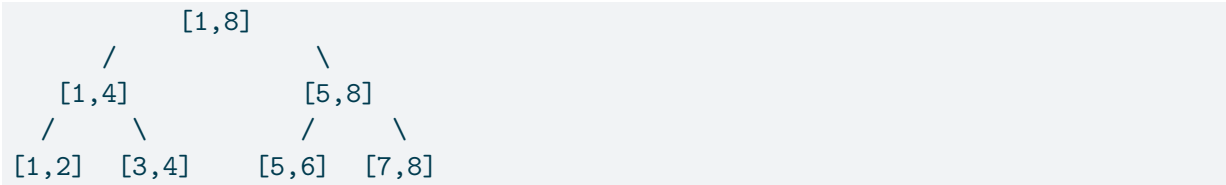
2026-07-06

Table of contents

1	Segment trees	3
2	Point update and range query	4
3	Rich node states	5
4	Lazy propagation	6
5	Why the maximum-subarray state works	7
6	Lazy propagation example	8
7	Mistakes to avoid	9
8	Local exercises	10
9	How to use the checker	11

1 Segment trees

A segment tree stores information about intervals. Each node owns a segment, and its value is computed by merging its two children.



If the merge is associative, such as `min`, `max`, `sum`, `gcd`, or a custom dynamic-programming state, a range query can be decomposed into $O(\log n)$ tree nodes.

2 Point update and range query

For point assignment, update one leaf, then recompute ancestors on the path to the root. This costs $O(\log n)$. Range minimum queries use `merge = min`.

Example with 1-based array:

```
a = [7, 2, 6, 3]
```

Leaves store:

```
[1,1]=7  [2,2]=2  [3,3]=6  [4,4]=3
```

Internal minimum nodes:

```
[1,2]=min(7,2)=2  
[3,4]=min(6,3)=3  
[1,4]=min(2,3)=2
```

If `a[2]` becomes 8, only nodes on the leaf-to-root path change:

```
[2,2], [1,2], [1,4]
```

This is why updates are logarithmic.

3 Rich node states

Segment trees are powerful because a node can store more than one number. For maximum subarray sum, each node stores:

```
sum    = total sum of the segment
pref   = best non-empty prefix sum
suff   = best non-empty suffix sum
best   = best non-empty subarray sum
```

To merge left L and right R:

```
sum    = L.sum + R.sum
pref   = max(L.pref, L.sum + R.pref)
suff   = max(R.suff, R.sum + L.suff)
best   = max(L.best, R.best, L.suff + R.pref)
```

This is a recurring red-level idea: design exactly the information that makes two halves composable.

4 Lazy propagation

If an update covers a whole node segment, do not immediately walk into all leaves. Store a pending tag on the node. For range-add and range-sum:

```
seg[node] += delta * segment_length  
lazy[node] += delta
```

Before visiting children, push the tag to them. Lazy propagation turns range update plus range query from $O(n)$ to $O(\log n)$.

5 Why the maximum-subarray state works

When two neighboring segments are merged, the best subarray in the union is one of exactly three types:

```
entirely in left  
entirely in right  
crosses the boundary
```

The crossing case must be:

```
best suffix of left + best prefix of right
```

That is why the node stores `pref` and `suff` in addition to `best`.

6 Lazy propagation example

Suppose a node covers $[5,8]$ and stores sum 20. Adding +3 to the whole range changes the sum by:

```
4 positions * 3 = 12  
new sum = 32
```

We record `lazy += 3` and stop. If a later query needs $[5,6]$, we push the tag to children before descending:

```
left child [5,6] gets +3 on 2 positions  
right child [7,8] gets +3 on 2 positions
```

The parent tag is then cleared.

7 Mistakes to avoid

Mixing inclusive and half-open ranges.

Forgetting that maximum subarray is non-empty in this section.

Applying lazy delta to `seg[node]` without multiplying by segment length.

Not pushing lazy tags before descending into a partially covered node.

8 Local exercises

- A. Range Minimum Updates: Point-assign values and answer range minimum queries.
- B. Maximum Subarray Updates: After each point assignment, print the maximum non-empty subarray sum of the whole array.
- C. Lazy Range Add Sum: Support range add updates and range sum queries.

9 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/37_segment_trees/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.